CMSC 436: Artificial Intelligence

Fall 2026, Instructor: Dr. Milos Manic,

http://www.people.vcu.edu/~mmanic ***Project 3***

CMSC 436: Artificial Intelligence
Project No. 3
Due 28th October, 2026, noon

Student certification:
Team member 1:
Print Name: __Gladys Ayaou_____ Date: __10/26/2026_____ I have contributed by doing the following:__Pr. 3.1 Understand and explore Reinforcement Learning_____ Signed: _____GA_____ (you can sign/scan or use e-signature)

Team member 2:
Print Name: _____Gavin Walker_____ Date: __10/27/2026_____ I have contributed by doing the following:_____Extra credit_____ Signed: _____GW_____ (you can sign/scan or use e-signature)

Team member 3:
Print Name: _____Edwina Sesay_____ Date: __10/27/26_____ I have contributed by doing the following:_____Pr. 3.2 _____ Signed: _____ES_____ (you can sign/scan or use e-signature)

Team member 4:
Print Name: ___Wyatt Henderson_____ Date: ____10/28/26_____ I have contributed by doing the following:_____P3.2 q 2 and 3_____ Signed: _____WH_____ (you can sign/scan or use e-signature)

Pr. 3.1 Understand and explore Reinforcement Learning (6pts)

1. What is Reinforcement Learning (RL)? Discuss. (2pts)
2. What are the real-world applications which use RL, discuss 2 application areas. (2pts)
3. Discuss the advantages and disadvantages of RL. (2pts)

Question 1: What is Reinforcement Learning (RL)? Discuss.

Reinforcement learning is a type of machine learning where a self-governing agent learns through trial and error, using numerical values to reward or penalize its actions to determine the optimal decisions over time. This type of learning works in a continuous loop between the agent and its environment, beginning with data which provides information about the current state of the environment. From there, the agent uses that information to choose an action (decision), which then transitions into a new state where it receives a reward or penalty based on the action taken. This process continues as the agent learns from its previous actions and the feedback it receives. Essentially, it creates a balance between trying and learning unknown behaviors and using previously learned or known behaviors to make the most optimal decisions based on what it knows.

Question 2: What are the real-world applications which use RL? Discuss two application areas.

Two very straightforward real-world applications that use reinforcement learning are healthcare and robotics.

Healthcare: In the healthcare field, researchers use reinforcement learning to study how AI can help determine things like medication dosages and treatment decisions. A specific example of this would be diabetes treatment, where the agent could recommend insulin doses based on the patient's changing health conditions. The environment would involve the patient's response to treatment, while the state would include patient information, such as blood glucose levels. In this case, a reward would be the patient's blood glucose remaining within a safe medical standard, while a penalty would be levels falling outside that healthy standard. Through this process, the agent learns which treatment decisions produce better outcomes over time. These recommendations would still need to be evaluated and monitored by healthcare professionals to ensure patient safety.

Robotics: Reinforcement learning can also be used to train robots to perform certain tasks through trial and error. For example, if we wanted to train a robot to approach a table and pick up a glass of water, the agent would be the robot being trained, and the environment would be its surroundings. The state would include the robot's current position, movement, and the location of the glass. The actions would involve how the robot approaches the table, which route it takes, and how it uses its arms or hands to pick up the glass. A reward would be successfully picking up the glass and bringing it back, while penalties could include knocking over the glass, failing to pick it up, or taking an inefficient route. Things like the robot losing balance or tipping over could also be

part of the trial-and-error process. Through repeated training, the robot can learn which movements produce successful outcomes and, over time, improve its ability to complete these tasks independently.

Question 3: Discuss the advantages and disadvantages of Reinforcement Learning.

When it comes to reinforcement learning, we think that sometimes the advantages can outweigh the disadvantages, but we also believe that this is case-specific and depends on the situation in which it is being applied.

One advantage of reinforcement learning is that it learns through experience. Reinforcement learning allows an agent to continuously learn through trial and error rather than being limited to one definite answer or a set of predefined decisions. This allows for more possibilities and adaptability. Another advantage is improved decision-making over time. Since reinforcement learning works in a continuous loop, the agent can use what it has learned from previous actions to make better decisions and produce better outcomes. We also think another advantage is its ability to solve complex problems, especially those involving multiple factors that need to be considered. For example, in this project's navigation scenario, factors such as toll roads, highways, local routes, and travel time could all influence which route is considered the most optimal.

As for disadvantages, we would say that reinforcement learning can be very time-consuming. An agent may require a large amount of training experience and numerous attempts, whether using the same environment or different situations, before it learns an effective strategy. This can also make it less cost-effective because training may require significant computing power and resources, including electricity and, depending on the computing infrastructure, as we all know, water for cooling. Another disadvantage is the possibility of mistakes during trial and error, especially when applied to real-world situations such as healthcare. This is why we think having a second step of human evaluation is important, because the agent's decisions are limited by what it has learned and the information available to it.

A good example of both an advantage and disadvantage would be the robotics application we previously discussed. An advantage is that the robot can gradually learn the best way to perform a task, such as picking up a glass of water. It can learn which route, movements, and positioning of its arms or hands work best for successfully grabbing the glass. However, a disadvantage is that the robot might knock over the glass 50 or even 1,000 times before successfully picking it up. Even after succeeding, that does not guarantee that it will successfully complete the task every time afterward. Essentially, this goes back to our original point that the advantages and disadvantages of reinforcement learning can be case-specific.

Pr. 3.2 Simulation of a self-driving cab using Q Learning (9 pts)

In this exercise you will learn:

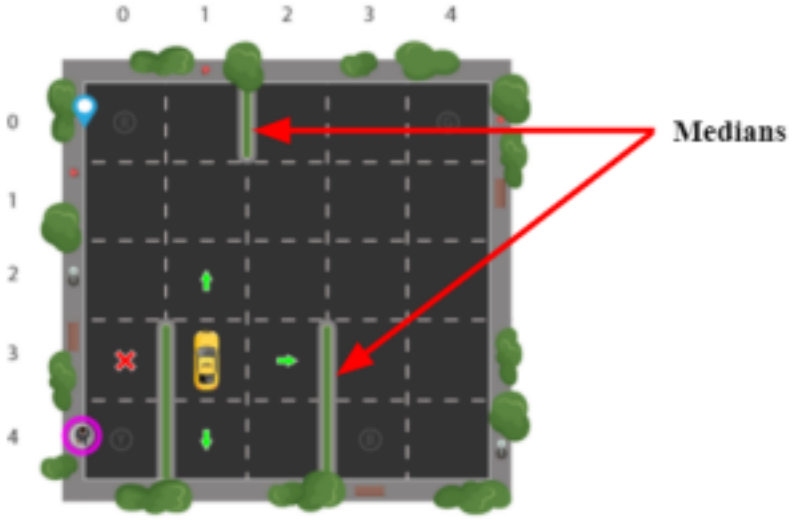
1. How to run **Google Collab**;
2. The importance of reinforcement learning and how Q Learning can be used in solving a real world problem.
3. The importance of hyperparameters in reinforcement learning.

Problem description: In this exercise you will design a simulation of a self-driving cab using **Q Learning**. Consider an intelligent agent called "SmartCab". A SmartCab picks up the passengers at one location and drops them off at another. A SmartCab performs the following tasks: 1. Drops off the passenger to the right location.

2. Performs route optimization to minimize the ride time.
3. Observe traffic rules and prioritize passenger's safety.

There are four locations labeled by letters. SmartCab picks up the passengers at one location and drops them off at another. The SmartCab receives +20 points for a successful drop-off and receives -1 point for every time-step it takes. There is also a -10 point penalty for illegal pick-up and drop-off actions.

The legal pick up and drop off locations are labeled as R, G, Y, B. (see the figure below). The SmartCab cannot drive through medians. The vehicle cannot go diagonally.



source: https://www.google.com/

There are different aspects to be considered while modeling an RL approach to this problem: 1. Rewards; 2. States, 3. Actions.

1. **Rewards**

   Since the SmartCab is reward-motivated and will learn by trial and error, here are some considerations to keep in mind when it comes to **rewards** and **penalties**:

   - The maximum number of time steps the agent can make is 8 (smaller city, see figure

below).

- The SmartCab should receive a high positive reward (+20) for a successful dropoff, because this behavior is highly desired.
- The SmartCab should be penalized (-10) for illegal actions.
- The SmartCab should be awarded a slight negative reward (-1) for each time step it takes to get to the destination. The smallest number of negative rewards deters SmartCab from taking routes longer than necessary (yes, this award is not ideally defined).

2. State Space

In RL, the agent encounters a state, and then takes action according to the state it is in. The **State Space** is the set of all possible situations (states) our SmartCab could be in. The state should contain useful information the agent needs to make the best next action.

3. Action Space

The allowable actions are: 1) move in a certain direction; 2) pick up a passenger, 3) drop off a passenger.

Do the following:

1. Change the following hyperparameters of the RL pipeline. Make a note of the reward for each combination of the hyperparameters. Discuss how the values of the hyperparameters affect the total reward for the smart cab **(6pts)**
   a. alpha: 0.01, 0.1, 0.9
   b. gamma: 0.40, 0.99
   c. epsilon: 0.5, 1

   Example: You can try:
   - alpha: 0.01
   - gamma: 0.40
   - epsilon: 0.5

   (then change alpha for the same gamma and epsilon, etc.)

2. Discuss the importance of reducing the epsilon parameter in achieving a higher reward **(1pts)**

3. Run the "*Smart_cab_brute_force.ipynb*" and compare and discuss the results with the reinforcement learning results. **(2pts)**

SmartCab Q-Learning Simulation

CMSC 436 • Project 3 • Pr. 3.2 – Question 1

Results

The following table summarizes the average rewards from 12 hyperparameter combinations. Each model was trained for 15,000 episodes and evaluated over 500 test episodes.

Run	Alpha	Gamma	Epsilon	Average reward
1	0.01	0.40	0.5	-94.31
2	0.01	0.40	1.0	-56.01
3	0.01	0.99	0.5	-22.79

4	0.01	0.99	1.0	-36.80
5	0.10	0.40	0.5	6.17
6	0.10	0.40	1.0	7.87
7	0.10	0.99	0.5	7.90
8	0.10	0.99	1.0	7.85
9	**0.90**	**0.40**	**0.5**	**8.03**
10	0.90	0.40	1.0	7.90
11	0.90	0.99	0.5	8.02
12	0.90	0.99	1.0	8.00

Best observed result: Run 9 — alpha = 0.90, gamma = 0.40, epsilon = 0.5; average reward = 8.03.

Discussion

For this part of the project, we tested different hyperparameters to see how they affected SmartCab's performance using Q-Learning. We changed alpha, gamma, and epsilon across 12 combinations and compared the average rewards after training for 15,000 episodes and testing for 500 episodes.

Changing the hyperparameters made a noticeable difference. When alpha was 0.01, all four combinations produced negative average rewards. The lowest was -94.31 at alpha = 0.01, gamma = 0.40, and epsilon = 0.5. This suggests the SmartCab learned too slowly to make consistently good decisions within the training period.

When we increased alpha to 0.1, all four average rewards became positive, ranging from 6.17 to 7.90. At alpha = 0.9, the rewards were consistently close to 8. The highest average reward was 8.03, using alpha = 0.9, gamma = 0.40, and epsilon = 0.5.

Gamma and epsilon also affected the results, although not in a consistent direction. For example, at alpha = 0.01 and epsilon = 0.5, increasing gamma from 0.40 to 0.99 improved the reward from -94.31 to -22.79. At alpha = 0.9, however, both gamma values produced very similar results.

Overall, alpha had the clearest impact in this set of experiments. Increasing the learning rate was associated with substantially better average rewards, while the effects of gamma and epsilon depended on the other settings.

What the Results Mean

The low learning rate of 0.01 performed poorly, while 0.9 consistently produced high rewards. All 12 experiments reported an average of zero illegal-action penalties during testing. However, the negative rewards in some runs suggest the taxi was taking too many steps or failing to complete trips within the step limit.

These results come from one set of randomized training runs. Because 8.03 and 8.02 are extremely close, we cannot say that the 8.03 combination will always perform better without repeating the experiments.

Conclusion

Our best observed combination was alpha = 0.9, gamma = 0.40, and epsilon = 0.5, with an average reward of 8.03. The experiment shows that hyperparameter choices can strongly influence how well SmartCab learns to complete trips.

2. Discuss the importance of reducing the epsilon parameter in achieving a higher reward (1pts)

- Reducing epsilon can be important since it controls the amount of exploration in Q Learning. With a higher epsilon the SmartCab will try random actions more frequently, while with a lower epsilon it's encouraged to use actions that can be more effective. If epsilon is reduced after sufficient training then the total reward is improved by allowing the SmartCab to take more efficient routes. However, reducing epsilon too early can prevent the SmartCab before it discovers better strategies, so a balance is needed between the two.

3. Run the "Smart_cab_brute_force.ipynb" and compare and discuss the results with the reinforcement learning results. (2pts)

- In the random action baseline, the cab reached 200 timestep limit and incurred 58 penalties without completing the task. Compared with the Q Learning, which learns from the rewards and penalties to improve its actions, random action selection is more inefficient and does not reliably complete the passenger drop off task.

Extra Credit: (3pts)

Implement the *epsilon_decay* as:
*epsilon_decay = min_epilson +(max_epilson - min_epilson) * exp (-epsilon_decay_rate * episode)*

using the following hyperparameters:

- min_epilson
- max_epilson
- epsilon_decay_rate

1. Which values of these hyperparameters (alpha, gamma, epsilon, min_epilson, max_epilson, epsilon_decay_rate) result in the highest reward?
2. What is the highest reward?

Note:

1. Project deliverable should be a zip file containing:
    a. Written report (pdf) with answers to all of the questions above.
    b. Updated Source code in python notebook file (.ipynb)
2. Submit your zip file to Canvas. Please name the zip file as GroupName_Project3.zip.